

CPU Scheduling, Deadlocks & Synchronization - Detailed Notes

DETAILED COMPREHENSIVE NOTES

Subject: Operating Systems | Chapter: Chapter 2: CPU Scheduling, Deadlocks & Synchronization | Academic Year: 2026-2027

1. CPU Scheduling Algorithms & Metrics

CPU Scheduling allocates CPU time among ready processes to optimize performance metrics: CPU Utilization, Throughput, Turnaround Time (Completion - Arrival), Waiting Time (Turnaround - Burst), and Response Time.

FCFS (First-Come First-Served): Non-preemptive, suffers from Convoy Effect (short processes queue behind long process).

SJF (Shortest Job First): Provably optimal for minimizing average waiting time; preemptive version is Shortest Remaining Time First (SRTF). Requires future burst estimation.

Round Robin (RR): Preemptive time-sharing using a Time Quantum (q). If q is too large, behaves like FCFS; if too small, context switch overhead degrades throughput.

2. Process Synchronization & Critical Section Problem

The Critical Section is a segment of code where shared resources are accessed. Any valid synchronization solution must satisfy three criteria: Mutual Exclusion, Progress, and Bounded Waiting.

Peterson's Algorithm provides a software solution for two processes using flags and turn variables.

Semaphores: Integer variables accessed via atomic operations wait() (P) and signal() (V). Counting semaphores control access to a finite pool of resources; Binary semaphores act as mutex locks.

3. Deadlocks & The Four Coffman Conditions

A deadlock is a situation where a set of processes are blocked because each holds a resource and waits for another resource held by another process in the set.

The Four Necessary Conditions for Deadlock: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait.

Banker's Algorithm: Avoids deadlock by testing safety state before granting resource requests using Allocation, Max, Available, and Need matrices ($\text{Need} = \text{Max} - \text{Allocation}$).